

Agents

- A* - Alice
- B* - Browser with Session storage, IndexedDB, SubtleCrypto API, and libsodium and libcrux libraries
- M* - Webauthn authenticator (passkey manager)
- S* - Quick Crypt server

Browser and Libsodium Functions

- C* - Constant-time comparator from libsodium
 $C(\text{value1}, \text{value2})$
- D_H* - BLAKE2b-512 KDF from libsodium
 $D_H(\text{subkey number}, \text{purpose}, \text{key})$
- D_P* - PBKDF2-HMAC-SHA512 key derivation FIPS-180-4 from SubtleCrypto
 $D_P(\text{material}, \text{salt}, \text{iterations})$
- D_S* - HMAC-SHA-512 from SubtleCrypto
 $D_S(\text{data}, \text{key})$
- E_a* - Symmetric cipher using AEAD algorithm *a*
 $E_a(\text{message}, \text{IV}, \text{additional data}, \text{key})$
a is one of:
 - 1. AES-256 in Galois/Counter mode from SubtleCrypto
 - 2. XChaCha20 with Poly1305 MAC from libsodium
 - 3. AEGIS 256 from libsodium
- G* - HMAC-SHA-512 key generator from SubtleCrypto
 $G()$
- H* - BLAKE2b keyed hash (MAC) generator from libsodium
 $H(\text{data} \dots, \text{key})$, each data argument hashed in order
- H₂* - SHA-256 from SubtleCrypto
 $H_2(\text{data})$
- PAD* - Appends 0 to 15 0xFF bytes to reach a multiple of 16, max 223
 $PAD(\text{data})$
- P_K* - ML-DSA-65 (FIPS 204) keypair generator from libcrux
 $P_K(\text{seed})$
- P_S* - ML-DSA-65 (FIPS 204) randomized signature generator from libcrux
 $P_S(\text{secret key}, \text{message}, \text{context})$
- P_V* - ML-DSA-65 (FIPS 204) signature verifier from libcrux
 $P_V(\text{public key}, \text{message}, \text{signature}, \text{context})$
- R* - Cryptographic pseudo-random generator from libsodium
 $R(\text{bits})$
- V* - Constant-time hash (MAC) validator from libsodium
 $V(\text{data}, \text{key}, \text{tag})$

Notes: *D_H* must be collision resistant for its key argument, not merely pseudorandom, or two cipher keys could share one *k_C*.

Cipher Variables

a	- Symmetric AEAD cipher and mode: [1, 2, 3]
ad_F	- Additional data included in ciphertext
b	- Valid or invalid MAC tag
cd	- Cipher data
cd_0	- Cipher data block 0
cd_N	- Cipher data block N
cd_u	- u_c encrypted under session key k_L , both defined in u_c Encryption at Rest
e_l	- Extra key material length: 0 (fixed)
err	- Error message and exit
f	- Block flags: 1 (TERM) on the final block of a loop, else 0
h	- Password hint text
h_E	- Encrypted hint
h_El	- Encrypted hint length
i	- PBKDF2-HMAC-SHA512 iteration count: min 420,000 max 4,294,000,000
k_H	- 256 bit ephemeral hint cipher key
k_{M0}	- 256 bit ephemeral message block0 cipher key
k_{MN}	- 256 bit ephemeral message blockN cipher key
k_S	- 256 bit ephemeral MAC key
k_C	- 256 bit key commitment, the only D_H output stored in cd
k_Cl	- k_C byte length: 32
kp_B	- Key purpose text: "Blck_Key"
kp_C	- Key purpose text: "Cmit_Key"
kp_H	- Key purpose text: "Hint_Key"
kp_S	- Key purpose text: "Sign_Key"
l	- Payload length
le	- Loop end (1-16), stored as $le - 1$ in the high nibble of one byte
lp	- Loop count (1-16), stored as $lp - 1$ in the low nibble of that byte
m	- Clear text
m_0	- Clear text block 0
m_N	- Clear text block N
m_E	- Block of encrypted message
n_{IV}	- Pseudo random initialization vector
$n_{IV}l$	- n_{IV} bit length: [96, 192, 256]
n_{IVH}	- Hint initialization vector: $n_{IV}l$ bits
n_S	- 128 bit pseudo random salt
N	- Block number
p	- Password text
p_l	- p byte length
r	- 384 bits of pseudo random data
t	- 256 bit MAC tag
t_L	- Preceding 256 bit MAC tag, or the single byte 0x00 at the start of a loop
u_c	- 256 bit user credential in memory
v	- Cipher data version: 8

Notes: A bit range $x[i : j)$ includes bit i and excludes bit j . For fixed empty fields, only a length of 0 is included.

Byte layout: Field widths and byte order are given by the cipher data structure table on the [protocol page](#) and by the [Hex Fiend template](#).

Message Encryption by A

$A \xleftrightarrow{\text{webauthn}} B, M \xleftrightarrow{\text{webauthn}} S$
 $B : u_c = cd_u$ decryption under k_L
 $A \rightarrow B : m, i, le$
 $v = 8$
 $lp = 1$
 $LOOP : B$ compute
 $t_L = 0x00$
 $A \rightarrow B : p, h, a$
 $r = R(384)$
 $n_S = r[0 : 128)$
 $n_{IV} = r[128 : 128 + n_{IV}l)$
 $n_{IVH} = D_H(2, kp_H, H(n_S, a, v, lp, e_l, n_{IV}))$
 $k_H = D_H(1, kp_H, H(n_S, a, v, lp, e_l, u_c))$
 $k_S = D_H(1, kp_S, H(n_S, a, v, lp, e_l, u_c))$
 $k_{M0} = D_P(p_l \parallel p \parallel u_c \parallel a \parallel v \parallel lp \parallel e_l, n_S, i)$
 $k_C = D_H(1, kp_C, H(n_S, k_{M0}))$
 $m_0 \parallel \dots \parallel m_N = m$
 $cd = cd_0 \parallel \dots \parallel cd_N$
 $m = cd$
 $lp = lp + 1$
 goto LOOP if $lp \leq le$
 $A \leftarrow B : cd$

cd_0 Encryption by B

$h_E = E_a(PAD(h), n_{IVH}, \emptyset, k_H)$
 $h_E l = len(h_E)$
 $f = 1$ if $\boxed{\text{TERM}}$ else 0
 $ad_F = f \parallel a \parallel n_{IV} \parallel n_S \parallel i \parallel le \parallel lp \parallel h_E l \parallel h_E \parallel k_C l \parallel k_C$
 $m_E = E_a(m_0, n_{IV}, ad_F, k_{M0})$
 $l = len(ad_F \parallel m_E)$
 $t = H(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S)$
 $t_L = t$
 $cd_0 = t \parallel v \parallel l \parallel ad_F \parallel m_E$

cd_N Encryption by B

$r = R(384)$
 $n_{IV} = r[0 : n_{IV}l)$
 $f = 1$ if $\boxed{\text{TERM}}$ else 0
 $ad_F = f \parallel a \parallel n_{IV}$
 $k_{MN} = D_H(N, kp_B, H(n_S, a, v, lp, e_l, k_{M0}))$
 $m_E = E_a(m_N, n_{IV}, ad_F, k_{MN})$
 $l = len(ad_F \parallel m_E)$
 $t = H(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S)$
 $t_L = t$
 $cd_N = t \parallel v \parallel l \parallel ad_F \parallel m_E$

Message Decryption by A

$A \xleftrightarrow{\text{webauthn}} B, M \xleftrightarrow{\text{webauthn}} S$
 $B : u_c = cd_u$ decryption under k_L
 $A \rightarrow B : cd$
LOOP : B compute
 $t_L = 0x00$
 $cd_0 \parallel \dots \parallel cd_N = cd$
 $t, v, l, ad_F, m_E = cd_0$
 $f, a, n_{IV}, n_S, i, le, lp, h_{El}, h_E, k_{Cl}, k_C = ad_F$
 $k_S = D_H(1, kp_S, H(n_S, a, v, lp, e_l, u_c))$
 $b = V(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S, t)$
 if ! b :
 $A \leftarrow B : err$
 $t_L = t$
 B verify $k_{Cl} = 32$ and $i \geq 420,000$
 B verify $lp = le$ in the first loop and $lp - 1$ in each successive loop
 $n_{IVH} = D_H(2, kp_H, H(n_S, a, v, lp, e_l, n_{IV}))$
 $k_H = D_H(1, kp_H, H(n_S, a, v, lp, e_l, u_c))$
 $h = PAD^{-1}(E_a^{-1}(h_E, n_{IVH}, \emptyset, k_H))$
 $B \rightarrow A : h$
 $B \leftarrow A : p$
 $k_{M0} = D_P(p_l \parallel p \parallel u_c \parallel a \parallel v \parallel lp \parallel e_l, n_S, i)$
 $b = C(k_C, D_H(1, kp_C, H(n_S, k_{M0})))$
 if ! b :
 $A \leftarrow B : err$
 $m = m_0 \parallel \dots \parallel m_N$
 B verify the last decoded block had $f = 1$
 $cd = m$
 goto LOOP if $lp > 1$
 $A \leftarrow B : m$

m_0 Decryption by B

$m_0 = E_a^{-1}(m_E, n_{IV}, ad_F, k_{M0})$

m_N Decryption by B

B verify the preceding block had $f = 0$
 $t, v, l, ad_F, m_E = cd_N$
 $f, a, n_{IV} = ad_F$
 $b = V(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S, t)$
if ! b :
 $A \leftarrow B : err$
 $t_L = t$
 B verify a and v equal those of cd_0
 $k_{MN} = D_H(N, kp_B, H(n_S, a, v, lp, e_l, k_{M0}))$
 $m_N = E_a^{-1}(m_E, n_{IV}, ad_F, k_{MN})$

u_c Encryption at Rest Variables

a	- Symmetric AEAD cipher and mode: 2
ad_F	- Additional data included in ciphertext
b	- Valid or invalid MAC tag
cd_p	- u_c encrypted under PRF output p_R
cd_r	- u_c encrypted under recovery secret $u_r \parallel u_i$
cd_u	- u_c encrypted under session key k_L
cd_x	- Encrypted u_c cipher data: one of cd_u, cd_p, cd_r
e	- Extra key material: u_i (fixed)
e_l	- e byte length: 16 (fixed)
err	- Error message and exit
f	- Block flags: 1 (fixed)
h_{El}	- Encrypted hint length: 0 (fixed)
i	- PBKDF2 iteration count: 0 (fixed)
k	- 256 bit master key: one of k_L, p_R , or $u_r \parallel u_i$
k_{Cl}	- Key commitment length: 0 (fixed)
k_{KS}	- HMAC-SHA-512 non-extractable CryptoKey, stored in IndexedDB and replaced at each login
k_L	- 256 bit per login master key
k_M	- 256 bit ephemeral message cipher key
k_S	- 256 bit ephemeral MAC key
kp_M	- Key purpose text: "Cphr_Key"
kp_S	- Key purpose text: "Sign_Key"
l	- Payload length
le	- Loop end: 1 (fixed)
lp	- Loop count: 1 (fixed)
m_E	- Encrypted message
n_{IV}	- Pseudo random initialization vector
$n_{IV}l$	- n_{IV} bit length: 192
n_S	- 128 bit pseudo random salt
p_R	- 256 bit passkey PRF output
$pkId$	- Passkey credential identifier
r	- 320 bits of pseudo random data
s	- IndexedDB slot label: "user-cred-key"
t	- 256 bit MAC tag
u_c	- 256 bit user credential (plaintext)
u_i	- User id
u_r	- 128 bit recovery id
v	- Cipher data version: 8

Format: At rest data uses the same wire format as messages, with the fields above fixed as listed. For cd_r this puts u_i in both k and e , which keeps the derivation identical across all three cd_x .

k_L Derivation by B

$B \leftarrow S : pkId$
 $k_{KS} = G()$
 $k_L = D_S(s \parallel pkId, k_{KS})[256 : 512)$

u_c Encryption Under k by B

where $(k, cd_x) \in \{(k_L, cd_u), (p_R, cd_p), (u_r \parallel u_i, cd_r)\}$
 $r = R(320)$
 $n_S = r[0 : 128)$
 $n_{IV} = r[128 : 128 + n_{IV}l)$
 $k_M = D_H(0, kp_M, H(n_S, a, v, lp, e_l, e, k))$
 $k_S = D_H(1, kp_S, H(n_S, a, v, lp, e_l, e, k))$
 $ad_F = f \parallel a \parallel n_{IV} \parallel n_S \parallel i \parallel le \parallel lp \parallel h_E l \parallel k_C l$
 $m_E = E_a(u_c, n_{IV}, ad_F, k_M)$
 $l = len(ad_F \parallel m_E)$
 $t = H(v \parallel l \parallel ad_F \parallel m_E \parallel 0x00, k_S)$
 $cd_x = t \parallel v \parallel l \parallel ad_F \parallel m_E$

cd_x Decryption Under k by B

where $(k, cd_x) \in \{(k_L, cd_u), (p_R, cd_p), (u_r \parallel u_i, cd_r)\}$
for $cd_u, cd_p : B \leftarrow S : u_i$
 $t, v, l, ad_F, m_E = cd_x$
 $f, a, n_{IV}, n_S, i, le, lp, h_E l, k_C l = ad_F$
 $k_S = D_H(1, kp_S, H(n_S, a, v, lp, e_l, e, k))$
 $b = V(v \parallel l \parallel ad_F \parallel m_E \parallel 0x00, k_S, t)$
if $!b :$
 $A \leftarrow B : err$
 $k_M = D_H(0, kp_M, H(n_S, a, v, lp, e_l, e, k))$
 $u_c = E_a^{-1}(m_E, n_{IV}, ad_F, k_M)$

Authentication Variables

ao	- Authentication options, including o, ch
ar	- Authentication response, including signed ch
bd	- HTTP request body
cd_p	- u_c encrypted under PRF output p_R
cd_r	- u_c encrypted under recovery secret $u_r \parallel u_i$
cd_u	- u_c encrypted under session key k_L
ch	- 256 bit challenge value
ch_R	- 256 bit recovery challenge value
ck	- Session cookie, a signed JWT
cs	- CSRF token
cw	- Alice's webauthn authenticator credentials
kp_R	- Key purpose text: "RecovKey"
kp_U	- Key purpose text: "UCredKey"
m_R	- Recovery proof message
m_U	- userCred proof message
$meth$	- HTTP request method
n_P	- 256 bit proof nonce value
o	- Quick Crypt origin "https://quickcrypt.org"
p_R	- 256 bit passkey PRF output
p_S	- Fixed 256 bit PRF salt, as base64url T89fTIZ37DRI-udjI_MBqc39I4yCyWJWHndLuYpD8yE
pk_R	- ML-DSA-65 recovery proof public key
pk_U	- ML-DSA-65 userCred proof public key
pth	- HTTP request path
qs	- HTTP request query string, omitted when empty
ro	- Registration options, including o, ch, u_i
rq	- Authorized request, including $meth, pth, bd, qs$
rr	- Registration response, including signed ch
sc_R	- Recovery signature context: "qcrypt/recovery/recover/v1"
sc_U	- userCred signature context: "qcrypt/usercred/proof/v1"
sc_W	- Replace words signature context: "qcrypt/recovery/replace/v1"
sk_R	- ML-DSA-65 recovery proof secret key
sk_U	- ML-DSA-65 userCred proof secret key
ts_R	- Recovery proof timestamp in milliseconds
ts_U	- userCred proof timestamp in milliseconds
u_c	- 256 bit user credential in memory
u_i	- 128 bit user id guaranteed to be unique
u_n	- A's chosen user name
u_r	- 128 bit recovery id
Δ	- Maximum request timestamp skew: 120 seconds
δ	- Delimiter byte: \n
σ_R	- ML-DSA-65 recovery proof signature
σ_U	- ML-DSA-65 userCred proof signature

Encodings: Every field of m_U and m_R is ASCII text: u_i and n_P are base64url, $meth$ is uppercase, pth is percent encoded, ts_U and ts_R are decimal milliseconds, and $H_2(bd)$ is lowercase hex.

Webauthn verification: Where S verifies a webauthn signature it verifies all of: the signature against the stored credential, ch against the challenge S stored for that purpose and user, the origin and relying party id against o , and that the authenticator performed user verification. The ch returned alongside rr and ar is the lookup key for the stored challenge.

Registration by A

$A \rightarrow B : u_n$
 $B \rightarrow S : u_n, o$
 $S : u_i = R(128); ch = R(256); \text{store } u_i, ch$
 $B \leftarrow S : ro$
 $B \rightarrow M : ro, p_S$
 $A \rightarrow M : cw$
 M create and store passkey, sign ch
 $B \leftarrow M : rr, p_R$
 $B : u_r = R(128)$
 $B : (pk_R,) = P_K(D_H(1, kp_R, u_r \parallel u_i))$
 $B : u_c = R(256)$
 $B : cd_p = u_c$ encryption under p_R
 $B : cd_r = u_c$ encryption under $u_r \parallel u_i$
 $B : (pk_U,) = P_K(D_H(1, kp_U, u_c)); \text{store } pk_U$
 $B \rightarrow S : rr, u_i, ch, pk_R, pk_U, cd_p, cd_r$
 S verify webauthn signature; store $rr, pk_R, pk_U, cd_p, cd_r$; remove ch
 S create ck, cs
 $B \leftarrow S : u_i, u_n, cs, ck$
 $B : cd_u = u_c$ encryption under k_L ; store cd_u
 $A \leftarrow B : u_r \parallel u_i$ as BIP39

No-PRF fallback: If M returns no p_R , B sends rr, u_i, ch, pk_R only; S generates $u_c = R(256)$, stores it encrypted, derives $(pk_U,)$ from u_c , and returns u_c ; B creates and stores cd_u from u_c .

Account mode: An entire account is either PRF or no-PRF mode. Downgrade from PRF to no-PRF is not supported, and B and S require every passkey to match the account's mode.

Authentication by A

$B \rightarrow S : o[, u_i]$
 $S : ch = R(256); \text{store } ch$
 $B \leftarrow S : ao$
 $B \rightarrow M : ao, p_S$
 $A \rightarrow M : cw$
 M sign ch
 $B \leftarrow M : ar, p_R$
 $B \rightarrow S : ar, ch$
 S verify webauthn signature; remove ch
 S create ck, cs
 $B \leftarrow S : u_i, u_n, cd_p, cs, ck$
 $B : u_c = cd_p$ decryption under p_R
 $B : (pk_U,) = P_K(D_H(1, kp_U, u_c))$
 B verify pk_U matches stored pk_U
 $B : cd_u = u_c$ encryption under k_L ; store cd_u

No-PRF fallback: S returns u_c in place of cd_p ; B creates and stores cd_u from u_c .

First use: B stores pk_U the first time it authenticates an account and rejects any later authentication that derives a different one, so subsequent cd_p substitution fails authentication.

Authorization by B

$B : u_c = cd_u$ decryption under k_L
 $B : (pk_U, sk_U) = P_K(D_H(1, kp_U, u_c))$
 $B : ts_U = \text{current time}$
 $B : n_P = R(256)$
 $B : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts_U\delta \parallel n_P\delta \parallel H_2(bd)[\delta \parallel qs]$
 $B : \sigma_U = P_S(sk_U, m_U, sc_U)$
 $B \rightarrow S : rq, ck, cs, ts_U, n_P, \sigma_U$
 S verify ck, cs
 S verify ts_U within Δ
 $S : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts_U\delta \parallel n_P\delta \parallel H_2(bd)[\delta \parallel qs]$
 $S : b = P_V(pk_U, m_U, \sigma_U, sc_U)$
if !b :
 $B \leftarrow S : err$
 $A \leftarrow B : err$
 $B \leftarrow S : \text{rq response}$

For state changing requests: S additionally requires n_P to be unknown and stores it, rejecting replays.

Add Passkey by A

A is authenticated; requests authorized per Authorization by B
 $B \rightarrow S : o$
 $S : ch = R(256)$; store ch
 $B \leftarrow S : ro$
 $B : u_c = cd_u$ decryption under k_L
 $B \rightarrow M : ro, p_S$
 $A \rightarrow M : cw$
 M create and store passkey, sign ch
 $B \leftarrow M : rr, p_R$
 $B : cd_p = u_c$ encryption under p_R
 $B \rightarrow S : rr, ch, cd_p$
 S verify webauthn signature; store rr, cd_p ; remove ch

No-PRF fallback: B omits p_S and does not decrypt u_c or send cd_p ; S stores the new passkey without encrypted u_c .

Replace Recovery Words by A

A is authenticated; requests authorized per Authorization by B

B repeat Authentication of A (B policy, not checked by S)

B : $u_r = R(128)$

B : $(pk_R, sk_R) = P_K(D_H(1, kp_R, u_r \parallel u_i))$

B : ts_R = current time

B : $n_P = R(256)$

B : $m_R = u_i \delta \parallel ts_R \delta \parallel n_P$

B : $\sigma_R = P_S(sk_R, m_R, sc_W)$

B : $u_c = cd_u$ decryption under k_L

B : $cd_r = u_c$ encryption under $u_r \parallel u_i$

B $\rightarrow$ S : $pk_R, ts_R, n_P, \sigma_R, cd_r$

S verify ts_R within Δ

S : $m_R = u_i \delta \parallel ts_R \delta \parallel n_P$

S : $b = P_V(pk_R, m_R, \sigma_R, sc_W)$

if !b :

B $\leftarrow$ S : err

A $\leftarrow$ B : err

S verify n_P unknown; store n_P

S store pk_R, cd_r

A $\leftarrow$ B : $u_r \parallel u_i$ as BIP39

No-PRF fallback: B does not decrypt cd_u or send cd_r ; S stores pk_R without cd_r .

$A \rightarrow B : u_r \parallel u_i$ as BIP39
 $B : (pk_R, sk_R) = P_K(D_H(1, kp_R, u_r \parallel u_i))$
 $B : ts_R = \text{current time}$
 $B : n_P = R(256)$
 $B : m_R = u_i \delta \parallel ts_R \delta \parallel n_P$
 $B : \sigma_R = P_S(sk_R, m_R, sc_R)$
 $B \rightarrow S : u_i, ts_R, n_P, \sigma_R$
 S verify ts_R within Δ
 $S : m_R = u_i \delta \parallel ts_R \delta \parallel n_P$
 $S : b = P_V(pk_R, m_R, \sigma_R, sc_R)$
 if !b :
 $B \leftarrow S : err$
 $A \leftarrow B : err$
 S verify n_P unknown; store n_P
 $S : ch_R = R(256)$; store ch_R
 $B \leftarrow S : ch_R, cd_r$
 $B : u_c = cd_r$ decryption under $u_r \parallel u_i$
 $B : (pk_U, sk_U) = P_K(D_H(1, kp_U, u_c))$
 $B : ts_U = \text{current time}$
 $B : m_U = u_i \delta \parallel mth \delta \parallel pth \delta \parallel ts_U \delta \parallel ch_R \delta \parallel H_2(\emptyset)$
 $B : \sigma_U = P_S(sk_U, m_U, sc_U)$
 $B \rightarrow S : u_i, o, ch_R, ts_U, \sigma_U$
 S verify ts_U within Δ ; verify ch_R is the stored value; remove ch_R
 $S : m_U = u_i \delta \parallel mth \delta \parallel pth \delta \parallel ts_U \delta \parallel ch_R \delta \parallel H_2(\emptyset)$
 $S : b = P_V(pk_U, m_U, \sigma_U, sc_U)$
 if !b :
 $B \leftarrow S : err$
 $A \leftarrow B : err$
 S delete existing rr
 $S : ch = R(256)$; store ch
 $B \leftarrow S : ro$
 $B \rightarrow M : ro, p_S$
 $A \rightarrow M : cw$
 M create and store passkey, sign ch
 $B \leftarrow M : rr, p_R$
 $B : cd_p = u_c$ encryption under p_R
 $B \rightarrow S : rr, u_i, ch, cd_p$
 S verify webauthn signature; store rr, cd_p ; remove ch
 S create ck, cs
 $B \leftarrow S : u_i, u_n, cs, ck$
 $B : cd_u = u_c$ encryption under k_L ; store cd_u

No-PRF fallback: S returns u_c in place of cd_r ; B omits p_S and the cd_r decryption.
 After the new passkey, B sends rr, u_i, ch only and S stores it without encrypted u_c .